

A
Special Project Report
On
“ADVANCED BUG TRACKER AND TASK
MANAGEMENT SYSTEM ”

By
Rani Yadav(23STUCHH010308)

Under the guidance of
Dr. P. Meena Kumari
Senior Assistant Professor



Faculty of Science & Technology,
The ICFAI Foundation for Higher Education, Hyderabad
April, 2026



Faculty of Science & Technology
The ICFAI Foundation for Higher Education
Hyderabad

CERTIFICATE

This is to certify that the project entitled “**BUG TRACKER AND TASK MANAGEMENT SYSTEM USING DJANGO**” by **Rani Yadav (23STUCHH010308)** has been submitted in the partial fulfilment of the requirements for the award of degree of **Bachelor of Technology in Artificial Intelligence and Data Science** from **Icfai Foundation of Higher Education, Hyderabad**.

The results embodied in this project have not been submitted to any other University or Institution for the award of any degree or diploma.

Internal Guide

Name:

Head of the Department

Name:

DECLARATION

I hereby declare that the Project entitled “BUG TRACKER AND TASK MANAGEMENT SYSTEM USING DJANGO” is the work done during the period from January 2026 to April 2026 and is submitted in partial fulfillment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering from [Your College Name]. The results embodied in this project have not been submitted to any other University or Institution for the award of any degree or diploma.

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

DATE:

PLACE: Hyderabad

SIGNATURE OF THE CANDIDATE

ACKNOWLEDGEMENT

There are many people who helped me directly and indirectly to complete my project successfully. I would like to take this opportunity to thank one and all. First of all, I would like to express my deep gratitude towards my internal guide Dr. P. Meena Kumari, Department of Computer Science and Engineering, for her support in the completion of my project. I wish to express my sincere thanks to Dr. P. Pavan Kumar, Head of the Department, and also to our Director Dr. K. L. Narayana for providing the facilities to complete the project.

I would like to thank all our faculty and friends for their help and constructive criticism during the project period. Finally, I am very much indebted to my parents for their moral support and encouragement to achieve my goals.

Rani Yadav
(23STUCHH010308)

ABSTRACT

Managing software bugs and tasks has become increasingly challenging in recent times due to the growing complexity of software applications and collaborative development environments. Poor tracking of issues, ineffective communication, and reliance on multiple unconnected tools often result in delays in completing projects and decreased productivity. In several organizations, critical issues are either overlooked or not properly recorded, which ultimately impacts the quality of the software. Hence, there is a strong requirement for an efficient and centralized system that can handle bugs and tasks in an organized and dependable way.

This project introduces a Bug Tracker and Task Management System developed using the Django web framework. The system offers an integrated platform that enables users to create, assign, modify, and monitor bugs and tasks throughout their lifecycle. It includes important features such as role-based access control to distinguish between administrators and normal users, a comment feature to improve communication, and file attachment support to enhance understanding of reported issues. Furthermore, a notification system is incorporated to provide real-time updates whenever changes occur, such as task assignments, status modifications, or new comments.

The application also provides dashboard functionalities that present important information such as the total number of bugs, resolved and unresolved issues, task progress, and overdue activities. These insights assist both users and administrators in tracking performance and making better decisions. The system is developed with a user-friendly interface using Bootstrap, ensuring simplicity, responsiveness, and accessibility across various devices.

By combining all these features into a single platform, the system increases transparency, strengthens collaboration among team members, and improves overall efficiency in handling software development processes. This project highlights the practical use of Django, database handling through SQLite, and modern web technologies to create a scalable and efficient management system.

CONTENTS

Certificate

Declaration

Acknowledgement

Abstract

1. Introduction

1.1 Problem Definition

1.2 Objective of the Project

1.3 Limitations of the Project

2. Literature Survey

2.1 Existing System

2.2 Disadvantages of Existing System

2.3 Proposed System

3. Software Requirements Specification

3.1 User Requirements

3.2 Software Requirements

3.3 Hardware Requirements

3.4 Data Flow Diagram (DFD)

4. Design

4.1 Module Design and Organization

4.2 Use Case and Class Diagrams

4.3 Algorithms

5. Implementation & Results

5.1 Explanation of Key Functions

5.2 Method of Implementation

5.3 Output Screens

5.4 Result Analysis

6. Testing & Validation

6.1 Explanation of Key Functions

6.2 Method of Implementation

6.3 Project Deployment

6.4 Output Screens

6.5 Result Analysis

7. Testing and Validation

8. Conclusion and Future Scope

9. Bibliography

1. Introduction

In recent times, software development has expanded rapidly due to the widespread use of digital services such as web applications, enterprise systems, educational platforms, and social media networks. As a result, a large volume of data and functionalities are constantly being created, updated, and maintained within software systems. This continuous growth increases the chances of errors and defects, commonly referred to as bugs, which can negatively impact the performance, security, and reliability of applications. Effectively managing these bugs has become a critical challenge in the field of software development.

Within software systems, bugs may arise from programming mistakes, system malfunctions, or unexpected user interactions, often resulting in incorrect outputs or application crashes. These issues must be detected, monitored, and resolved promptly to ensure the smooth operation of the system. In addition, task management plays a vital role in development, as it involves organizing the work required to design, maintain, and enhance software applications. In many development setups, bugs and tasks are handled using emails, spreadsheets, or separate tools, making it difficult to track progress and maintain clear communication among team members. The Bug Tracker and Task Management System is developed to offer a centralized platform where users can report bugs, assign tasks, and monitor their progress efficiently. This system enhances collaboration, improves issue management, and increases the overall productivity of development teams.

1.1 Problem Definition

In several software development environments, bugs and tasks are handled through manual approaches such as emails, spreadsheets, or independent tools, making the process inefficient and difficult to monitor. When a bug is reported, it may not be properly recorded or assigned to the appropriate individual, resulting in delays in resolving the issue. Similarly, tasks may not be clearly defined or effectively tracked, leading to confusion among team members and incomplete assignments.

If proper tracking mechanisms are not in place, critical bugs may be ignored or remain unresolved for extended periods. This can negatively affect the performance and dependability of the software system and may result in user dissatisfaction. Furthermore, poor communication among team members can lead to duplication of efforts or improper task management. Without a well-structured system, it becomes challenging to maintain accurate records of all activities and track progress efficiently.

With the increasing complexity of software systems and the growing importance of teamwork, it is necessary to have a centralized platform that can manage bugs and tasks in an organized and systematic manner. However, developing such a system is not simple, as it involves managing user roles, task allocation, real-time updates, and data handling effectively. Therefore, there is a strong need for an efficient Bug Tracker and Task Management System that can streamline these processes and improve overall productivity.

1.2 Objective of Project

The effectiveness of a bug tracking and task management system largely depends on its ability to detect and resolve bugs accurately within a reasonable time frame. Several traditional approaches, such as manual tracking through emails and spreadsheets, have been used to manage bugs and tasks. However, these methods are not sufficiently efficient, as they lack proper tracking mechanisms and do not support real-time updates. As a result, many issues may remain unresolved or experience delays. Therefore, it is essential to manage bugs and tasks efficiently to ensure the smooth operation of software systems. An effective bug tracking system plays a key role in enhancing productivity and maintaining the overall quality of software.

1.3 Limitations of Project

Modern web-based systems have been introduced as alternatives to traditional methods of managing bugs and tasks, offering improved tracking, organization, and collaboration. These systems are more efficient compared to manual approaches such as emails and spreadsheets. However, despite these benefits, certain limitations must be considered. The system is mainly designed for small to medium-scale applications, and as the number of users and data grows, performance issues may arise. Managing a large volume of bugs, tasks, and user activities may require further optimization and a more robust database system. The current implementation uses SQLite, which is suitable for development and smaller applications but may not be ideal for large-scale production environments that demand high performance and scalability.

Another limitation is that the system relies heavily on user input for reporting bugs and tasks. If users provide incomplete, unclear, or incorrect information, it may affect the accuracy and efficiency of resolving issues. Furthermore, the system does not include advanced features such as real-time communication, integration with external tools, or automated bug detection, which could further enhance performance. It also offers limited support for advanced analytics and reporting, which may be necessary for improved decision-making in larger organizations. Security features can also be strengthened to better protect sensitive data. Although the system provides a well-structured and efficient approach to managing bugs and tasks, continuous improvements are required to make it more scalable, reliable, and adaptable to different environments.

2. Literature Survey

In recent years, software development has advanced rapidly due to the widespread availability of digital services such as web applications, enterprise systems, educational platforms, and social networking services [1]. As a result, a significant amount of data and functionalities are continuously created, updated, and maintained within software systems. This ongoing expansion increases the likelihood of errors and defects, commonly referred to as bugs, which can impact the performance, reliability, and security of applications. Efficient management of these bugs has become a major challenge in software development.

In many development environments, bugs and tasks are managed using manual approaches such as emails, spreadsheets, and separate tools, making it difficult to track issues and ensure proper communication among team members [2]. These methods often result in delays in resolving problems and reduce transparency in project workflows. Therefore, there is a strong need for an efficient system that can manage bugs and tasks in a structured and organized manner.

Modern bug tracking and task management systems are designed to offer centralized control, real-time updates, and improved collaboration among users. These systems contribute to increased productivity, accurate tracking of issues, and better quality in software development processes.

2.1 Existing System

In many software development environments, bugs and tasks are handled using traditional methods such as emails, spreadsheets, and basic tracking tools. When an issue is reported, it is manually recorded without proper organization or classification. This makes it difficult to monitor the status of bugs and determine who is responsible for resolving them. As a result, important issues may be delayed or overlooked, negatively affecting the performance and reliability of the software system [3]. With the growing complexity of software systems used in businesses, organizations, and various applications, it is essential to manage bugs and tasks efficiently from the early stages. However, managing these issues remains challenging due to the lack of effective communication, real-time updates, and centralized data handling [4].

Existing systems for bug tracking and task management mainly depend on manual processes or simple tools that do not provide advanced functionalities. These systems rely heavily on user input and basic tracking methods without proper automation or integration. Consequently, they are not efficient enough to support large-scale projects and dynamic development environments [5].

2.2 Disadvantages of Existing System

Effective management of bugs and tasks depends on accurate tracking and timely resolution. Various traditional methods such as emails, spreadsheets, and simple tools have been used for

this purpose. However, these approaches are not efficient, as they lack proper organization, real-time updates, and effective communication among team members [6]. One of the key disadvantages of existing systems is the absence of centralized data management. Since information is distributed across multiple platforms, it becomes difficult to maintain accurate records and monitor progress effectively. This may lead to duplication of work, missed issues, and delays in resolving bugs [7].

Additionally, these systems do not offer proper assignment mechanisms, making it difficult to identify responsibility for tasks. Another limitation is the lack of real-time notifications and collaboration features, which may prevent team members from staying updated about changes, thereby affecting coordination and productivity. Furthermore, these systems are not scalable and may fail to handle large volumes of data efficiently, making them unsuitable for complex software development environments [8].

2.3 Proposed System

As an improved alternative to traditional methods of managing bugs and tasks, an efficient web-based Bug Tracker and Task Management System has been developed to handle software issues effectively and enhance team collaboration. In recent years, modern web-based management solutions have become widely used, as they are more efficient, well-structured, and flexible compared to manual approaches such as emails and spreadsheets. The proposed system offers a centralized platform where users can report bugs, assign tasks, and monitor their progress in real time. It is built using the Django framework, which provides improved performance, security, and scalability. The system also includes role-based access control, enabling administrators and users to operate with different levels of permissions.

Advantages of Proposed System:

Accuracy: The system maintains precise tracking of bugs and tasks by keeping proper records, updating statuses, and assigning responsibilities. This reduces errors and enhances the reliability of software development processes.

Real-time Updates: The system delivers real-time notifications, allowing users to stay updated about changes such as task assignments, status modifications, and comments. This improves communication and coordination among team members.

Centralized Management: All bugs and tasks are stored in a single centralized database, making it easier to access, manage, and monitor information. This minimizes data duplication and improves overall organization.

Improved Collaboration: The system provides features such as comments and file attachments, enabling users to communicate efficiently and share relevant information related to bugs and tasks.

Scalability: Built with modern web technologies, the system can efficiently handle increasing amounts of data and users with further enhancements and optimizations.

Adaptability: The system can be easily modified and extended by adding features such as APIs, real-time communication, and advanced analytics, making it flexible to meet changing requirements.

Enhanced Performance: By automating bug tracking and task management processes, the system reduces manual effort and increases overall productivity and efficiency.

Overall, the proposed system delivers a more organized and efficient approach to managing bugs and tasks. It enhances transparency, improves communication, and provides better control over software development activities. However, future improvements such as advanced analytics, real-time communication, and integration with external tools can further strengthen the system and make it more scalable and robust.

4. Software Requirements Specification

The Bug Tracker and Task Management System is required to meet several user requirements to ensure its effectiveness, usability, and reliability. The system is developed to offer a centralized platform for efficiently managing bugs and tasks in a well-structured manner. It should support multiple users, provide secure access, and enable smooth interaction with the application. The primary goal of the system is to increase productivity, improve communication, and ensure accurate tracking of software issues and tasks.

User Requirements

A user-friendly interface is necessary, allowing users to easily create, manage, and track bugs and tasks without any complexity. Users should be able to navigate the system smoothly and clearly understand the displayed information. In addition, users may need detailed insights into the status of bugs and tasks, including their progress, priority levels, and assigned members. Scalability is important to efficiently manage an increasing number of users, bugs, and tasks. Users may also expect customization features such as filtering, searching, and sorting tasks based on different parameters. Finally, the system must ensure strong security by protecting user data and implementing proper authentication and authorization mechanisms.

Accuracy: The system should accurately track and manage bugs and tasks by maintaining proper records, status updates, and assignments.

Real-Time Updates: The system must provide real-time notifications for events such as task assignments, status changes, and comments to keep users informed.

User-Friendly Interface: The system should offer an intuitive interface that enables users to easily create bugs, assign tasks, and monitor progress.

Transparency: Users should have clear visibility of the status, priority, and details of bugs and tasks for better understanding and management.

Scalability: The system should be capable of handling a growing number of users and increasing data as the project expands.

Customization: The system should support features like filtering, searching, and sorting tasks based on criteria such as priority, status, or assigned users.

Security: The system must ensure secure management of user data through proper authentication and role-based access control to safeguard sensitive information.

4.1 Software Requirements

To achieve the objectives of the project, suitable software components are required. Python is selected as the primary programming language because of its simplicity and strong support for

web application development. The Django framework is utilized as the backend framework, offering built-in features such as authentication, ORM (Object Relational Mapping), and security functionalities. SQLite is used as the database to store all relevant data including users, bugs, tasks, comments, and notifications. The frontend is developed using HTML, CSS, and Bootstrap to create a responsive and user-friendly interface. JavaScript is also incorporated to improve interactivity and enhance the overall user experience. Libraries such as Pillow are used for handling file uploads and image processing within the application. Development tools like Visual Studio Code are used for coding and debugging purposes. Version control systems such as Git and platforms like GitHub are used to manage and maintain the project efficiently.

Programming Language: Python is used as the primary programming language due to its ease of use and strong support for web development through frameworks like Django.

Web Framework: Django is used for backend development, providing features such as URL routing, authentication, ORM, and built-in security mechanisms.

Database: SQLite is used as the default database to store all application data including users, bugs, tasks, comments, and notifications.

Frontend Technologies: HTML, CSS, Bootstrap, and JavaScript are used to design a responsive and interactive user interface.

Libraries and Tools: Pillow is used for handling files and images, along with additional Django libraries for authentication and request processing.

Development Environment: Visual Studio Code or similar IDEs are used for writing, testing, and debugging the application code.

Version Control: Git and platforms such as GitHub are used for version control and effective project management.

4.2 Hardware Requirements

To ensure the smooth operation of the Bug Tracker and Task Management System, appropriate hardware components are necessary. The system is designed to function efficiently on standard computing devices without requiring high-end hardware. The development and execution of this project were performed on an Acer One 14 laptop, which offers adequate performance for running the application effectively.

The system requires a computer equipped with a modern processor capable of handling application logic and database operations efficiently. A minimum of 4 GB RAM is needed for proper functioning, although higher memory can enhance performance when managing multiple tasks simultaneously. Sufficient storage space is also essential for storing project files, database records, and uploaded files such as images and documents.

The system is compatible with common operating systems such as Windows, Linux, and macOS. A stable internet connection is necessary for installing dependencies, downloading required libraries, and testing the web application. Since the system is lightweight, it does not require specialized hardware such as GPUs or high-performance servers, making it both cost-effective and easy to deploy.

Processor: Standard processor available in the Acer One 14 laptop, suitable for development and execution.

RAM: Minimum 4 GB RAM, as available in the Acer One 14 system.

Storage: Adequate storage space for project files, database, and media, typically 256 GB or more.

Device Used: Acer One 14 Laptop.

Operating System: Windows or any compatible operating system.

Internet Connectivity: Required for installation, updates, and testing.

4.3 DFD diagram of the Project

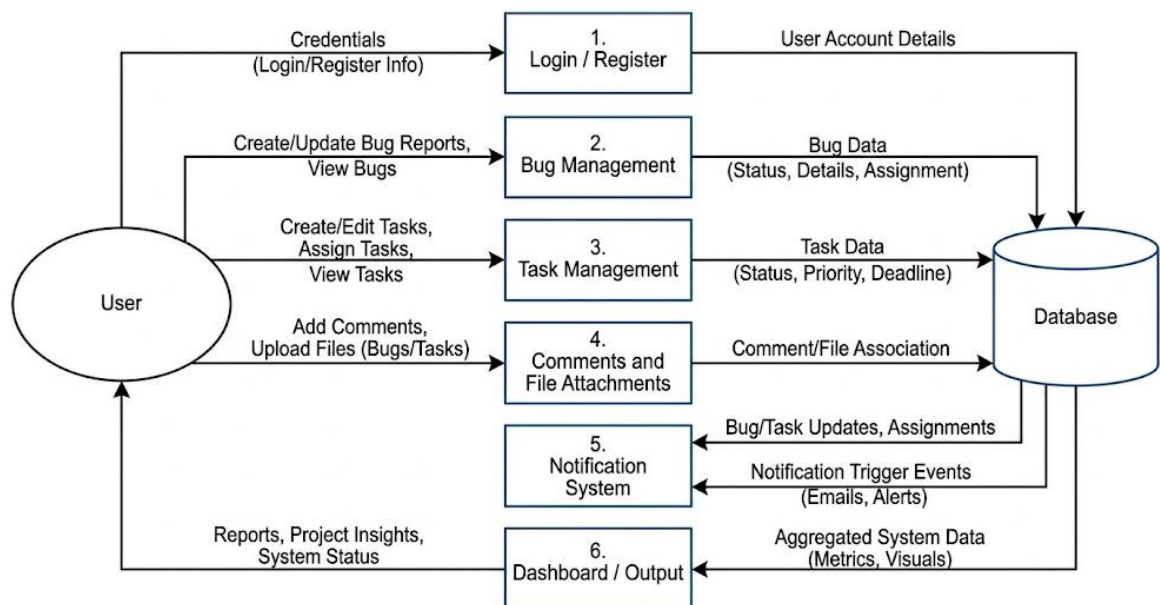


Figure 1: Data Flow diagram for the proposed model

The diagram shows how the user interacts with the Bug Tracker System to manage bugs and tasks. The system processes user inputs such as login, bug updates, and task management, and stores all data in the database. It also supports comments, file attachments, and notifications. The dashboard displays reports and system status based on stored data.

5. Design

The design phase of the Bug Tracker and Task Management System focuses on organizing the system into different modules and defining their interactions. The system is structured to provide a clear and well-organized workflow for efficiently managing bugs and tasks. The overall design is divided into multiple modules, including authentication, bug management, task management, comments and file attachments, notifications, and dashboard. Each module performs a specific role and works together to ensure the smooth functioning of the system. The application follows a client-server architecture, where users interact with the frontend interface, while the backend processes requests and communicates with the database. The Django framework is used to manage application logic, routing, and data handling. The design ensures that the system remains scalable, secure, and easy to maintain. Proper separation of concerns is maintained among different components, which enhances the overall performance and reliability of the application.

5.1 Module Design and Organization

Module 1: Authentication Module

- a. The system enables users to register by providing required details such as username, email, and password.
- b. The entered information is validated and securely stored in the database.
- c. Users can log in using their credentials, with authentication handled by Django's built-in system.
- d. After successful login, users are redirected to the dashboard to access system features.

Module 2: Bug and Task Management Module

- a. Users can create bugs and tasks by entering details such as title, description, priority, and status.
- b. Bugs and tasks can be assigned to specific users to ensure proper management and accountability.
- c. The system allows users to update, edit, and view bugs and tasks based on their current status.
- d. This module tracks the lifecycle of bugs and tasks, including stages such as open, in progress, and completed.
- e. All bug and task information is stored in the database and retrieved when required for display and updates.

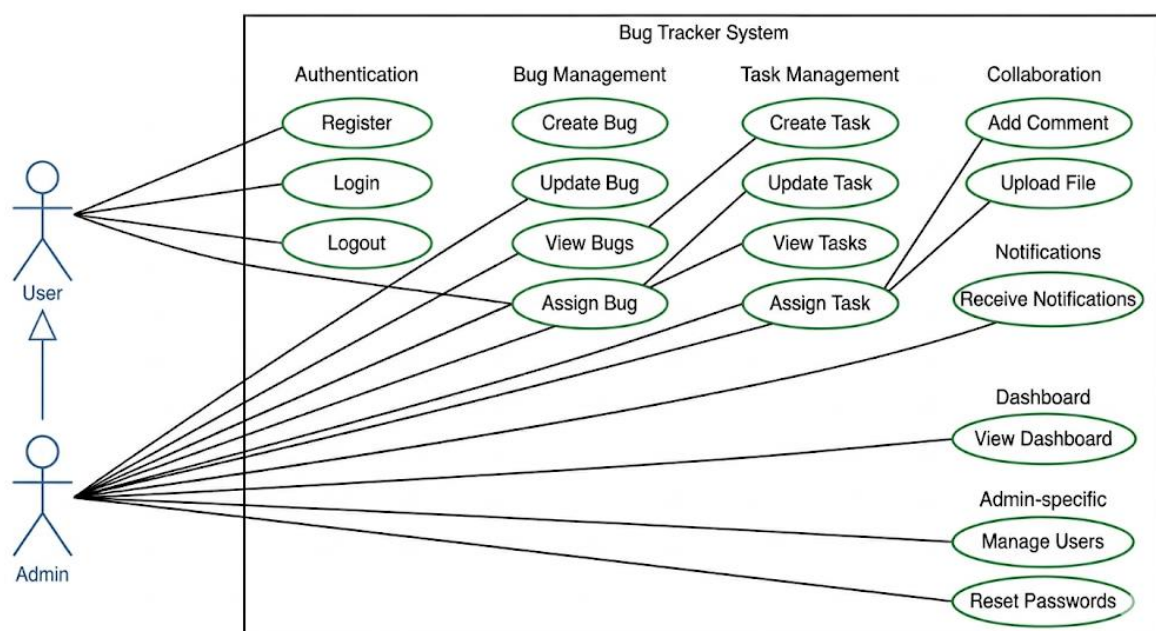
Module 3: Collaboration and Notification Module

- a. Users can add comments and upload files related to bugs and tasks to improve understanding and communication.
- b. The system generates notifications for key events such as task assignments, status changes, and comments.
- c. Notifications are stored in the database and displayed to users in real time.

- d. The dashboard presents summarized information such as total bugs, task progress, and overall system status.
- e. The output is displayed on the frontend in the form of reports, notifications, and project insights.

5.2 Use Case and Class Diagrams

In software engineering, use case diagrams are utilized to illustrate the interaction between users (actors) and the system. They assist in understanding the functional requirements and demonstrate how different users engage with various features of the system. In the Bug Tracker and Task Management System, the primary actors are the user and the administrator. The user can perform actions such as registration, login, creating bugs, managing tasks, adding comments, uploading files, and accessing the dashboard. The administrator has additional privileges, including managing users, assigning tasks, and monitoring overall system activities. These interactions are represented in the use case diagram using standard UML notation. Class diagrams are used to depict the structure of the system by showing different classes, their attributes, and the relationships between them. In this project, key classes include User, Bug, Task, Comment, Attachment, and Notification. Each class contains specific attributes and methods, and they are interconnected through relationships such as associations and dependencies. The use case and class diagrams help in visualizing the system architecture and provide a clear understanding of how various components are organized and interact with one another.



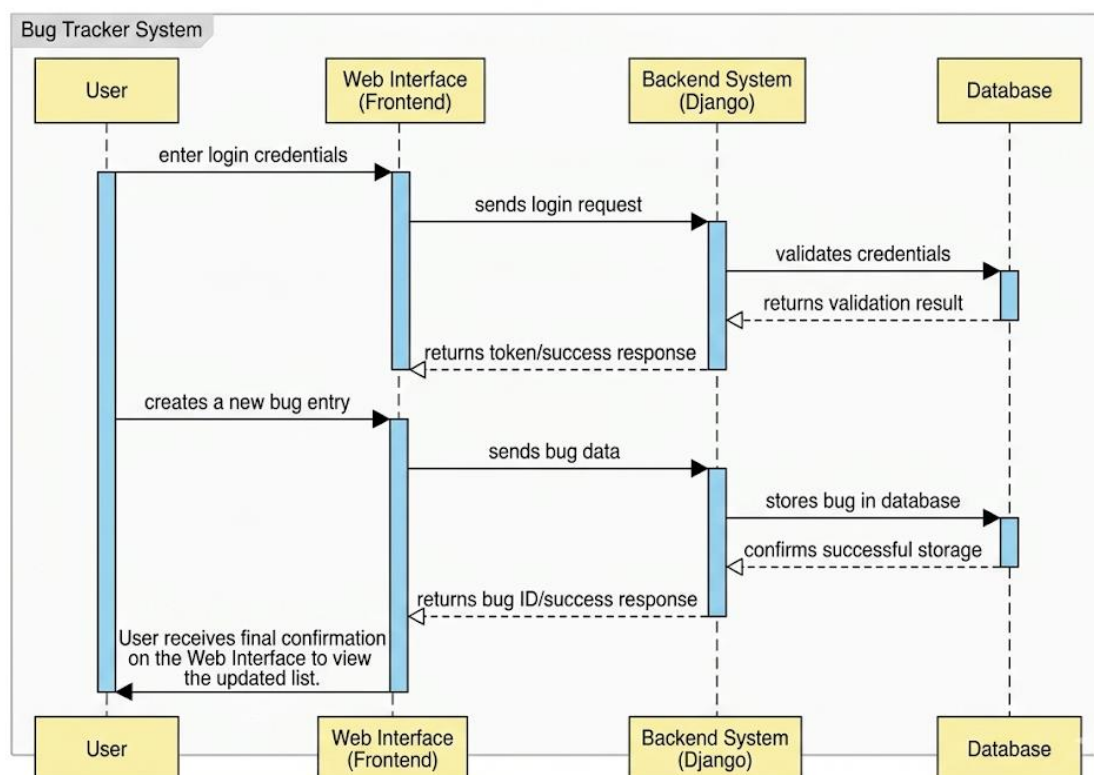
The diagram illustrates the interactions between users and the Bug Tracker System. Users can perform activities such as registering, logging in, managing bugs and tasks, adding comments, uploading files, and accessing the dashboard. The administrator has additional privileges,

including managing users and resetting passwords. The diagram explains how different features of the system are accessed and used by both users and administrators.

Sequence Diagram

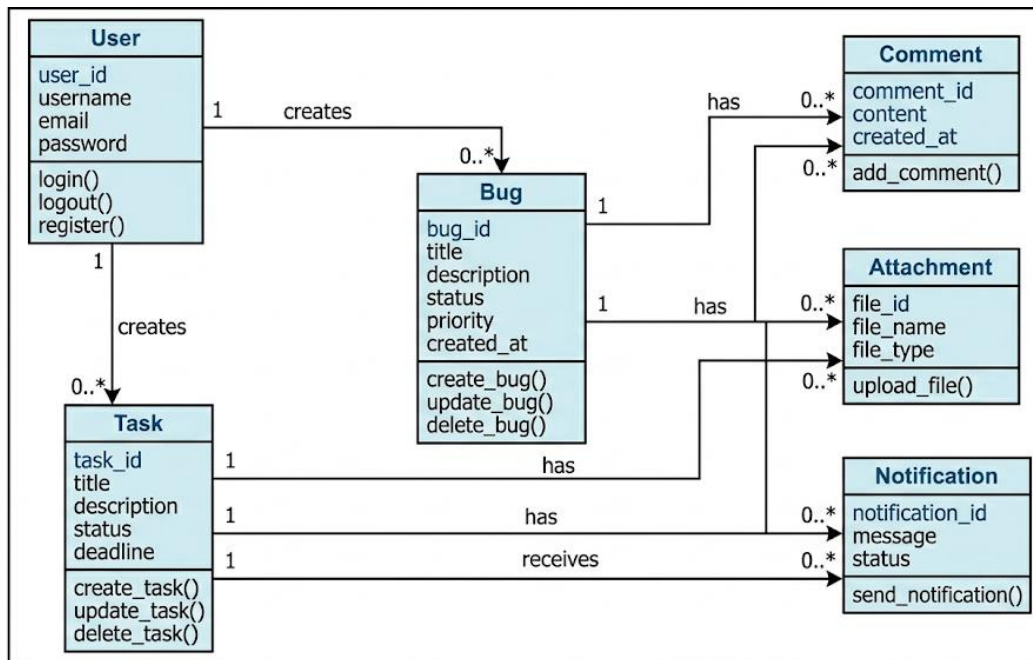
The sequence diagram illustrates the interaction between the user, frontend interface, backend system, and database over a period of time. Initially, the user enters login credentials, which are verified by the backend through the database. Once authentication is successful, the user proceeds to create a bug, and the system handles the request by storing the bug details in the database. The database then confirms the successful operation, and the system returns a response to the user, displaying the updated information.

Sequence Diagram: Bug Tracker System (College Project)



Class Diagram

The class diagram illustrates the structure of the Bug Tracker and Task Management System. It includes key classes such as User, Bug, Task, Comment, Attachment, and Notification. Each class consists of specific attributes and methods, and they are linked through relationships such as associations and dependencies. The diagram explains how various components of the system are organized and how they interact with one another.



5.3 Algorithms

In the Bug Tracker and Task Management System, various logical processes are implemented to manage user activities such as authentication, bug handling, task management, and notifications. The system follows structured algorithms to process user inputs and execute operations efficiently.

User Authentication

User authentication is used to confirm the identity of users before allowing access to the system. It verifies the entered credentials against the data stored in the database.

Functionality:

The system provides secure access by validating user credentials. If the entered details are correct, the user is granted access to the dashboard; otherwise, an error message is displayed. This process helps maintain security and prevents unauthorized access.

Bug Management

Bug management is responsible for handling the creation, updating, and tracking of bugs within the system. It enables users to report issues and monitor their progress effectively.

Functionality:

The system stores bug information such as title, description, priority, and status in the database. Users can update these details and track the progress of bugs. This ensures efficient issue handling and timely resolution.

Task Management

Task management is used to create, assign, and monitor tasks within the system. It helps in organizing work and increasing productivity.

Functionality:

The system allows users to manage tasks by updating their status, priority, and deadlines. Tasks are stored in the database and can be viewed or modified as needed. This ensures proper tracking and successful completion of tasks.

Notification System

The notification system is designed to inform users about important updates such as task assignments, bug updates, and comments.

Functionality:

The system generates notifications based on specific events and stores them in the database. These notifications are displayed to users in real time, helping them stay informed and improving communication.

Comment and File Handling

This process enables users to add comments and upload files related to bugs and tasks for better collaboration.

Functionality:

The system stores comments and file attachments in the database, allowing users to share additional details. This improves the understanding of issues and enhances teamwork.

6. Implementation and Results

The Bug Tracker and Task Management System is developed using Python as the primary programming language and the Django framework for backend development. Django offers a secure and reliable environment with built-in features such as authentication, ORM (Object Relational Mapping), and URL routing, which simplify the development process. The frontend is designed using HTML, CSS, Bootstrap, and JavaScript to create a responsive and user-friendly interface. SQLite is used as the database to store all essential data, including user information, bugs, tasks, comments, attachments, and notifications.

The system is implemented using a modular approach, where each module is developed and tested separately before being integrated. The authentication module ensures secure user registration and login functionality. The bug management module allows users to create, update, assign, and track bugs with attributes such as status, priority, and severity. Similarly, the task management module provides features for managing tasks, including tracking deadlines and monitoring progress.

The collaboration module enables users to add comments and upload files related to bugs and tasks, improving communication and providing better clarity of issues. The notification system generates alerts for important events such as task assignments, updates, and comments, ensuring users receive real-time information. The dashboard module presents a summary of system activities, including the number of bugs, task status, and overall progress.

The system is tested across various functionalities such as user authentication, bug creation, task assignment, and notification handling. The results indicate that the system performs efficiently and provides accurate tracking of bugs and tasks. It effectively reduces manual effort, improves communication among team members, and increases overall productivity.

The system output is displayed through a well-organized interface, allowing users to easily view and manage their bugs and tasks. The dashboard provides both graphical and numerical insights into system performance, helping users analyse their workload effectively. Overall, the implementation shows that the system is reliable, user-friendly, and suitable for managing software development processes in a structured manner.

6.1 Explanation of Key Functions

Step 1: Access the System

Step 2: Register/Login

Step 3: Create Bug/Task

Step 4: Processing Phase

Step 5: Display Output

Step 1: Access the System

Initially, the user needs to run the application and open the system in a web browser. The Bug Tracker System can be accessed using the provided URL. If the system is already running, the same URL can be used again to access the application.

Step 2: Register/Login

The user needs to open the system in a web browser, which displays the login page. If the user is new, they must register by entering the required details. Existing users can log in using their credentials.

Step 3: Create Bug/Task

After logging in, the user can add a new bug or task by providing details such as title, description, priority, and status. The user can also assign the bug or task to another user.

Step 4: Processing Phase

The system processes the provided data and stores it in the database. Users can update, modify, and track the status of bugs and tasks. Notifications are generated for important actions such as assignments or updates.

Step 5: Display Output

The processed data is displayed on the frontend interface. Users can view updated bug and task lists, notifications, and dashboard insights, which assist in tracking progress and managing work effectively.

6.2 Method of Implementation

Code for importing libraries

```
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.models import User
from .models import Bug, Task, Comment, Attachment, Notification
```

Code for User Authentication

```
user = authenticate(request, username=username, password=password)
if user is not None:
    login(request, user)
    return redirect('dashboard')
```

```
else:  
    return render(request, 'login.html', {'error': 'Invalid credentials'})
```

Code for Creating Bug

```
bug = Bug.objects.create(  
    title=title,  
    description=description,  
    priority=priority,  
    status='Open',  
    created_by=request.user  
)  
bug.save()
```

Code for Creating Task

```
task = Task.objects.create(  
    title=title,  
    description=description,  
    status='Pending',  
    assigned_to=user  
)  
task.save()
```

Code for Updating Data

```
bug.status = 'In Progress'  
bug.save()
```

Code for Adding Comment

```
comment = Comment.objects.create(  
    content=content,  
    user=request.user,  
    bug=bug  
)  
comment.save()
```

Code for Notification System

```
Notification.objects.create(  
    user=assigned_user,  
    message="New Task Assigned"  
)
```

Code for Displaying Results

```
bugs = Bug.objects.all()
tasks = Task.objects.all()

return render(request, 'dashboard.html', {
    'bugs': bugs,
    'tasks': tasks
})
```

6.3 Project Deployment

The Bug Tracker and Task Management System was successfully deployed on a cloud platform, making it accessible through the internet. Deployment allows users to interact with the system in a real-world environment instead of running it locally. The Django application was configured for production to ensure security and proper performance.

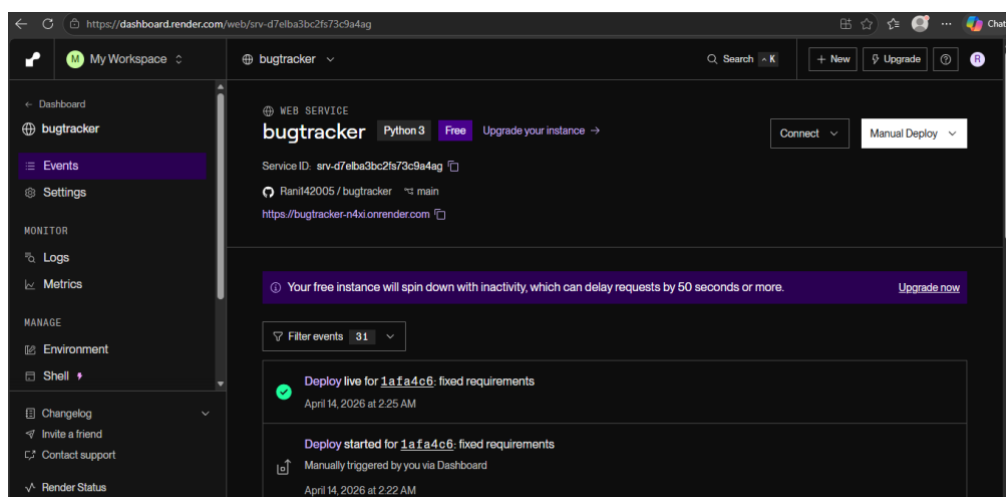
Deployment Steps:

- Disabled debug mode for security
- Configured ALLOWED_HOSTS
- Collected and managed static files
- Installed required dependencies
- Applied database migrations

The database was properly integrated to store all data such as users, bugs, tasks, and notifications. After deployment, the system was tested to ensure all features work correctly without errors.

Deployed Link:

👉 <https://bugtracker-n4xi.onrender.com>



6.4 Output Screens

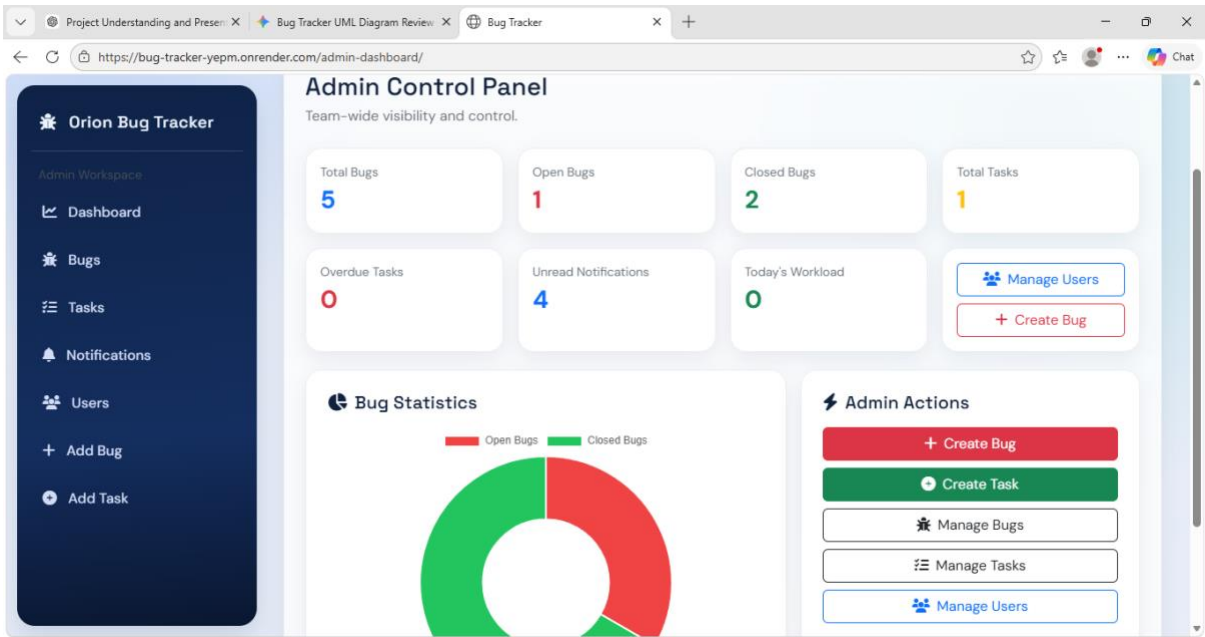


Fig: Admin dashboard

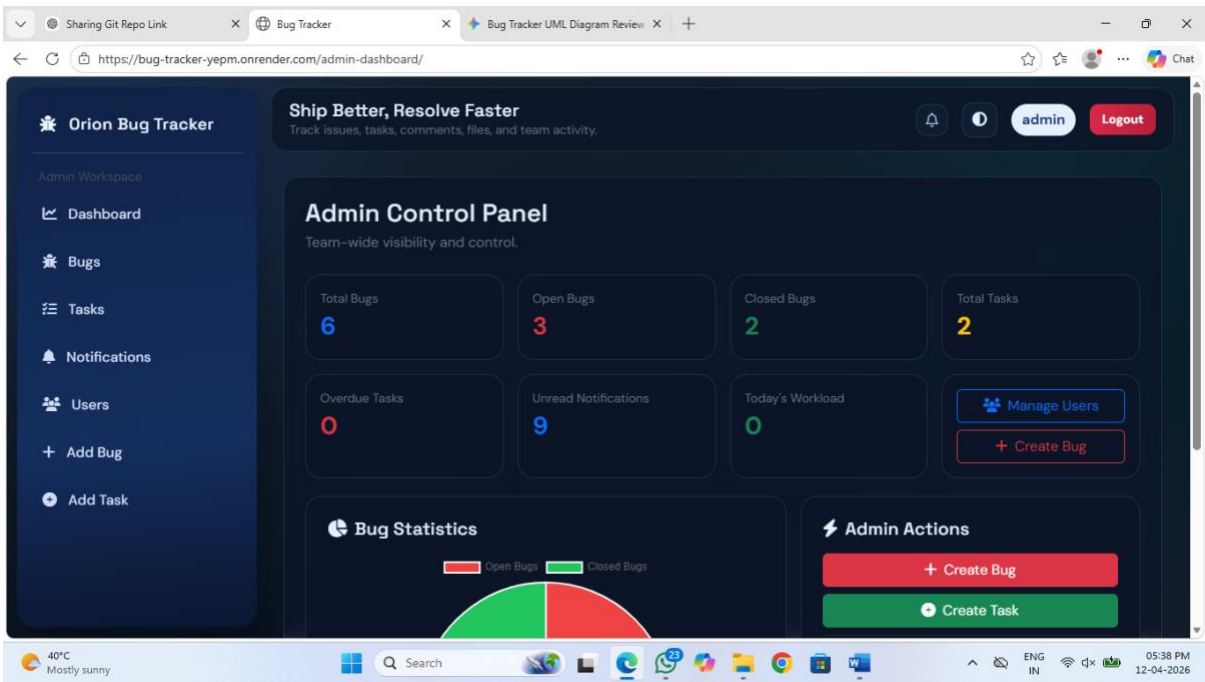


Fig: Dark mode enabled

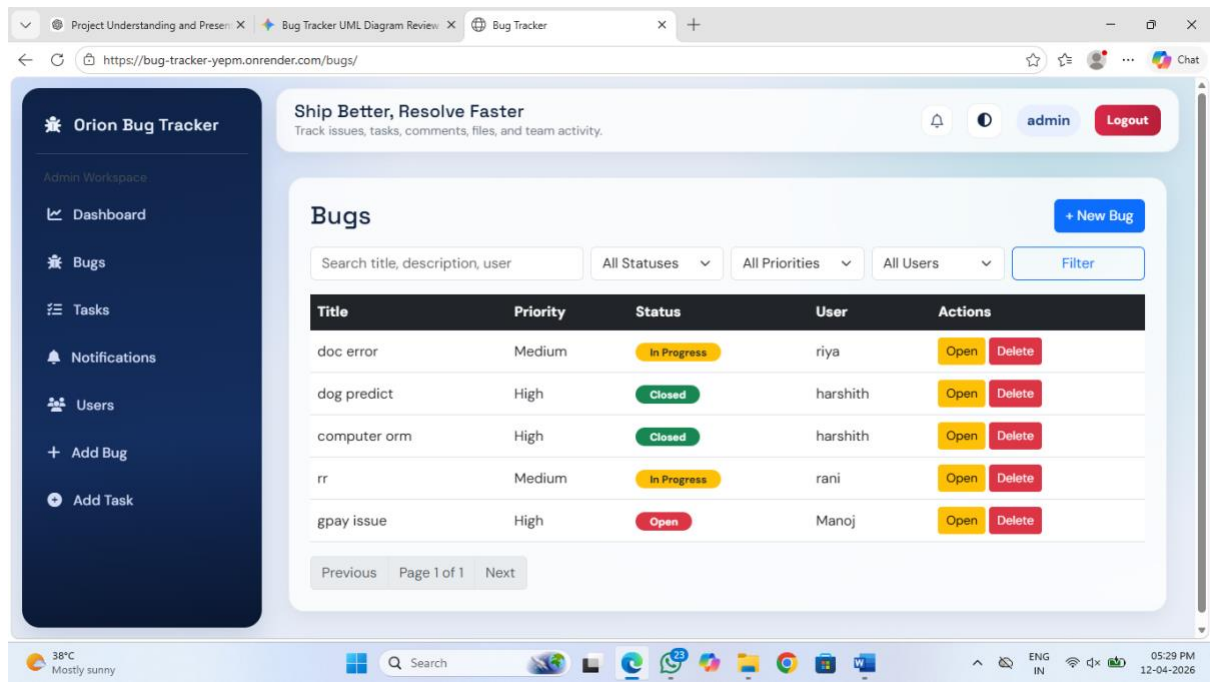


Fig: Bug list page

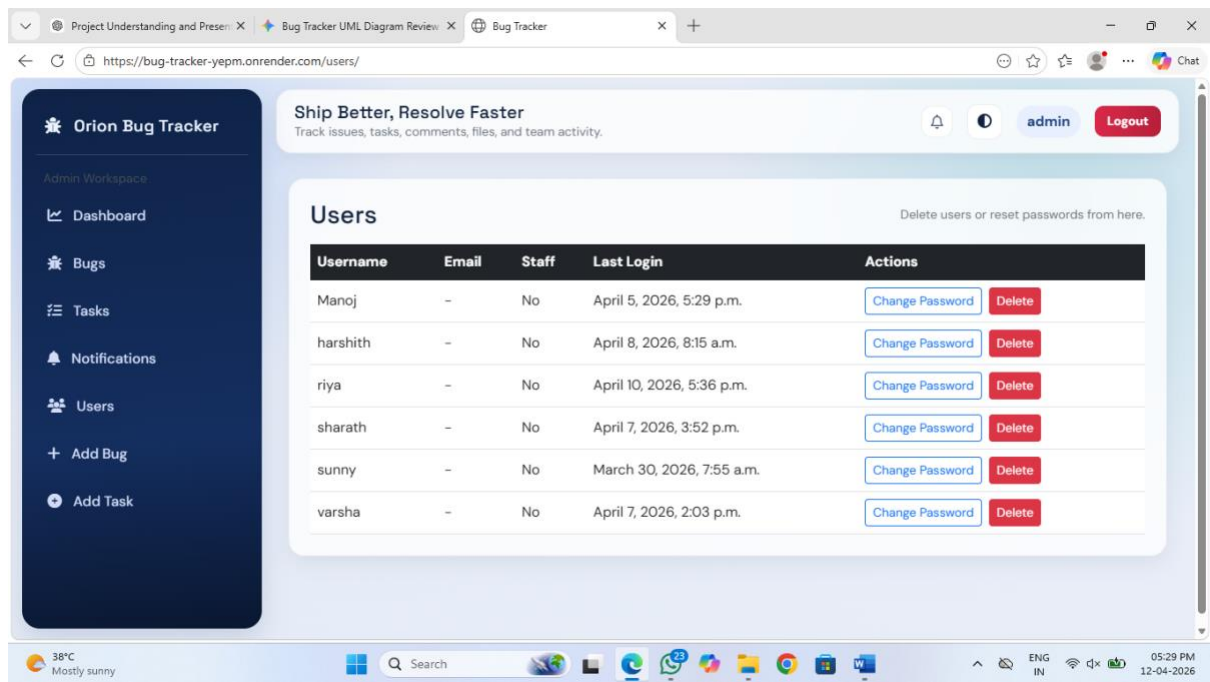


Fig: Users list page

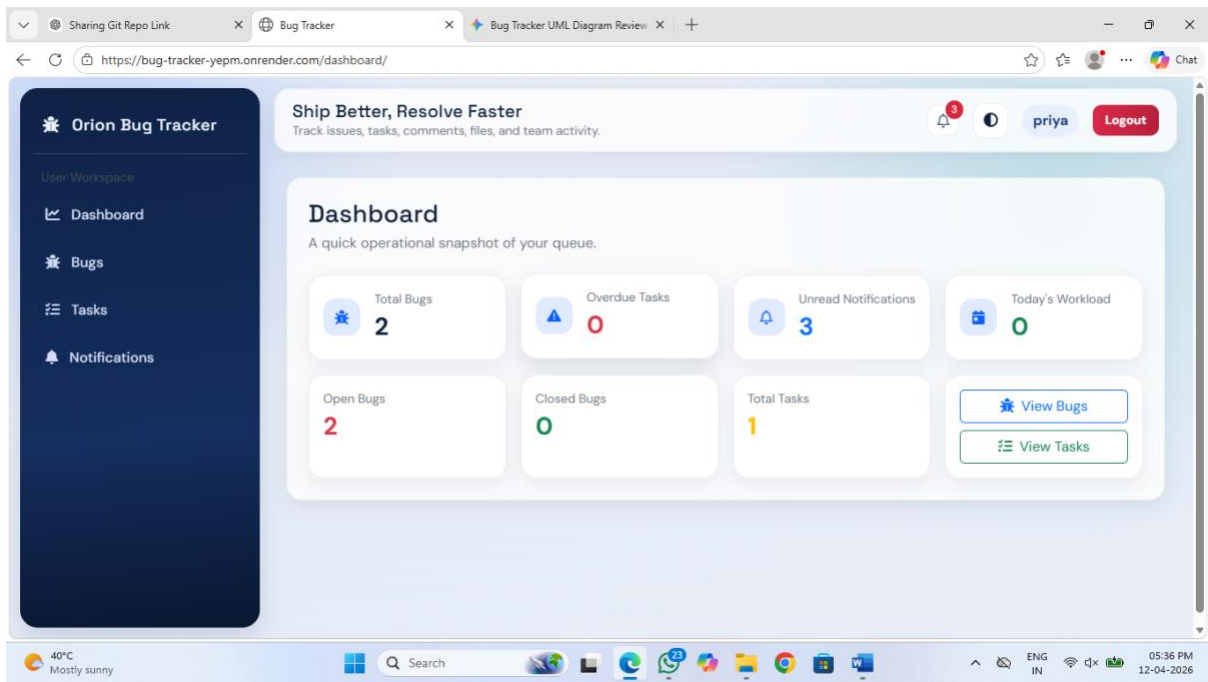


Fig: User dashboard

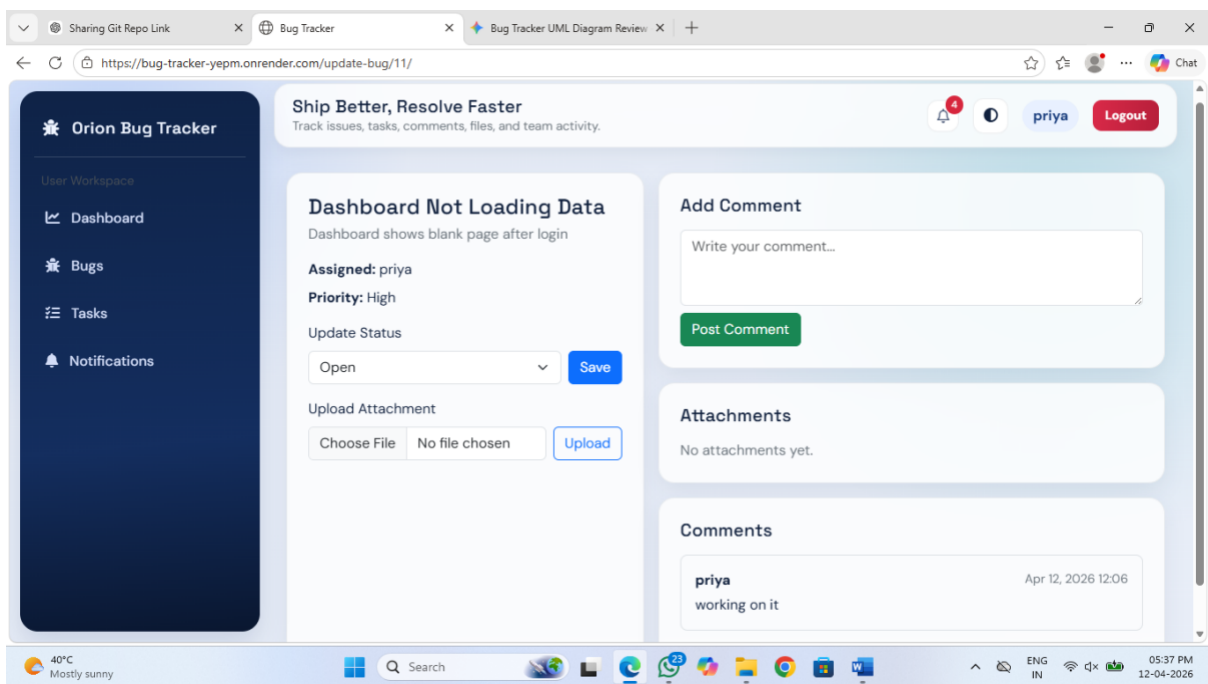


Fig: Bug page with comment feature

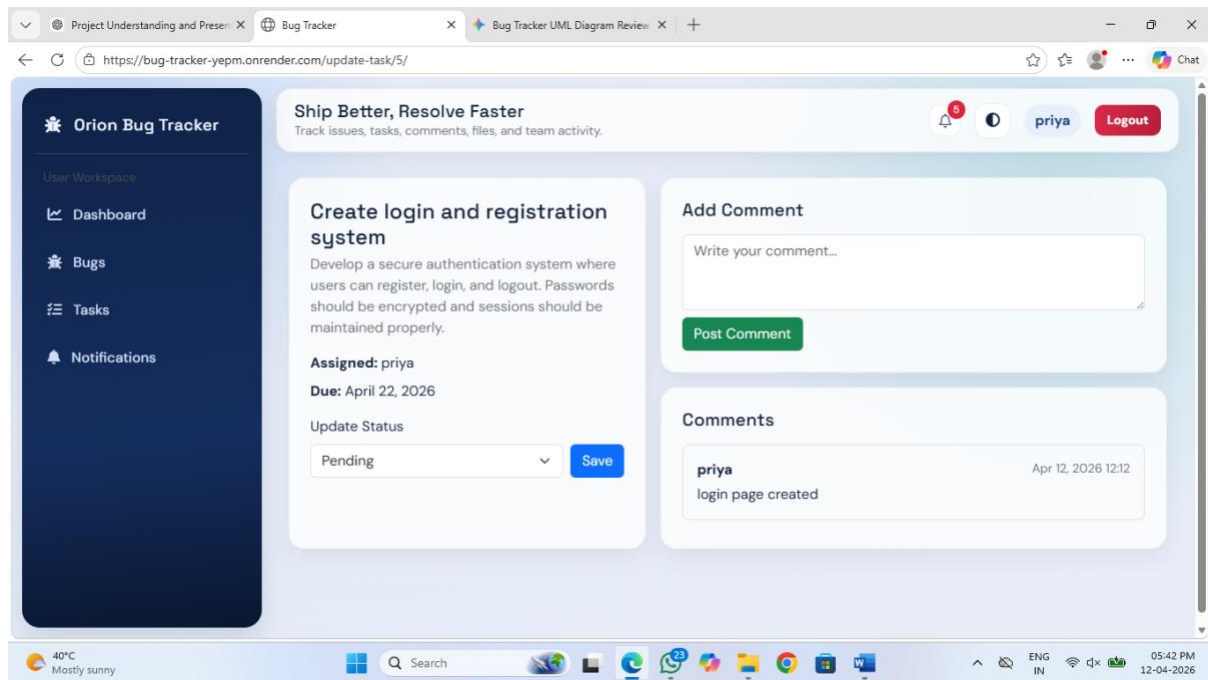


Fig: Task page with comment feature

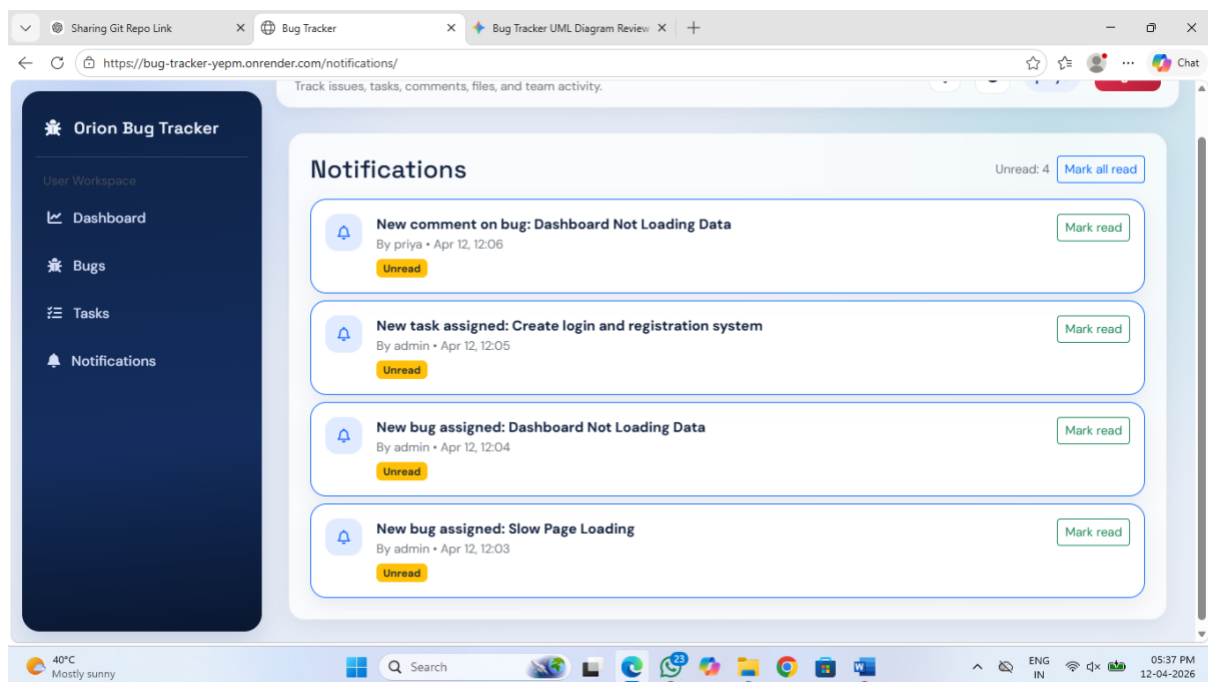


Fig: Notifications page

6.5 Result Analysis

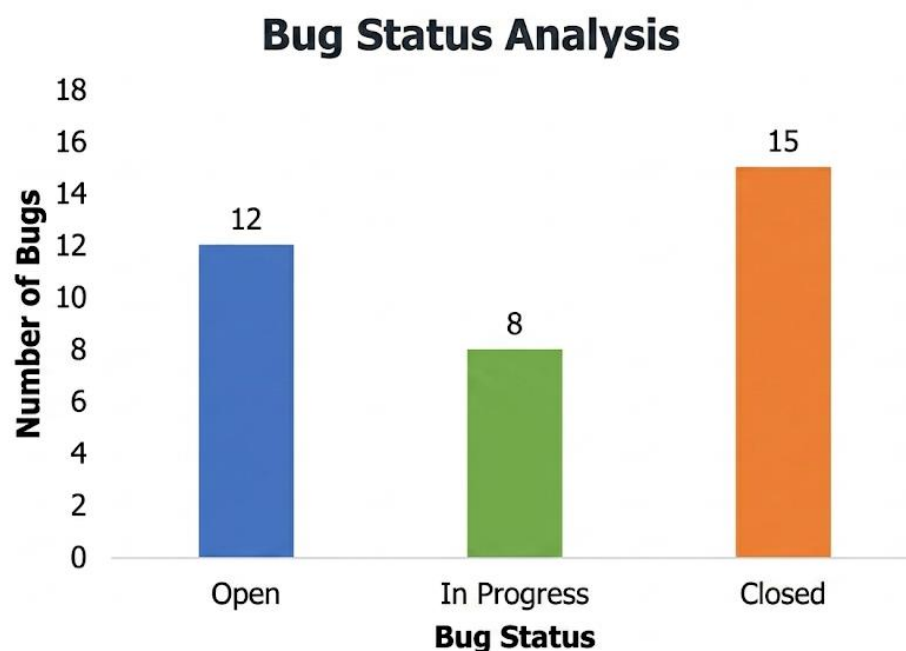
The Bug Tracker and Task Management System was evaluated for multiple functionalities, including user authentication, bug creation, task assignment, comments, file uploads, and notification generation. The system performed successfully in all test cases and delivered accurate and consistent results.

The authentication module properly verifies user credentials and ensures secure access to the system. The bug and task management modules enable users to create, update, assign, and track bugs and tasks efficiently. All data is securely stored and retrieved from the database without any loss or inconsistency.

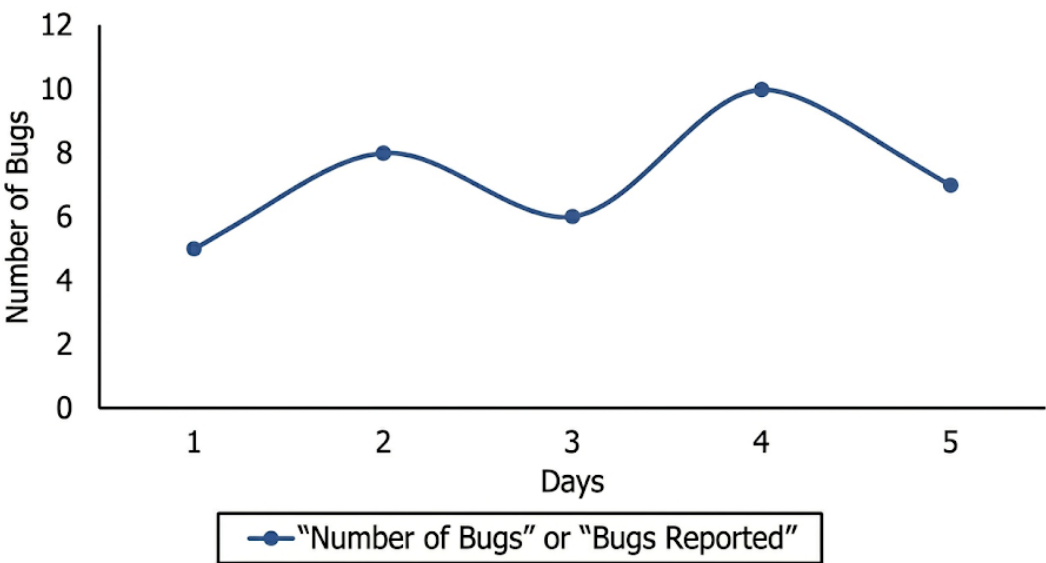
The notification system functions effectively by generating real-time alerts for important actions such as task assignments, updates, and comments. This keeps users informed and enhances communication within the team. The comment and file upload features improve collaboration by allowing users to share additional information related to bugs and tasks.

The dashboard presents a clear and well-organized overview of system activities, including total bugs, resolved and unresolved issues, task progress, and overdue tasks. These insights help users monitor their work and make informed decisions. The user interface is responsive and easy to use, ensuring smooth interaction with the system.

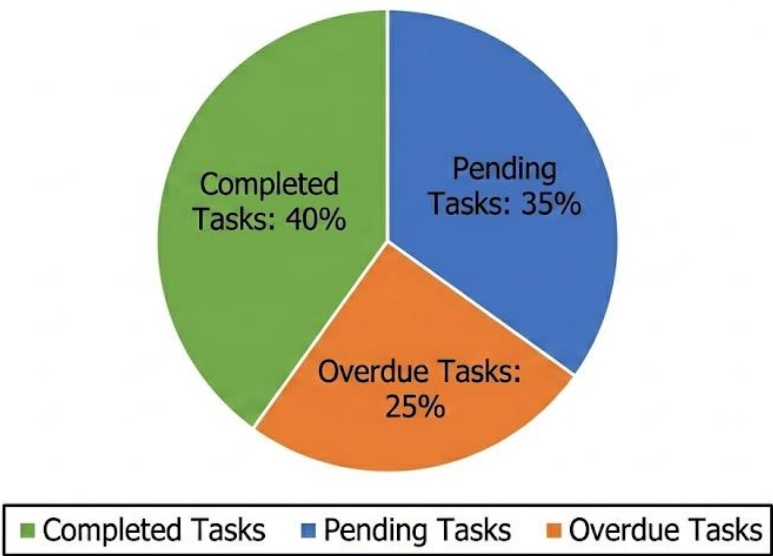
Overall, the system minimizes manual effort, increases efficiency, and improves coordination among team members. It ensures effective tracking of issues and tasks, resulting in higher productivity and improved software development processes. The results confirm that the system is reliable, user-friendly, and suitable for real-world applications.



Bug Trend Analysis



Task Status Distribution



7 Testing and Validation

UnitTesting

It is also referred to as component testing and involves testing individual modules or functions of the system to verify their correctness. In this project, unit testing is carried out on various modules such as authentication, bug management, task management, and the notification system. Each module is tested separately to ensure that it produces accurate output for the given inputs. Unit testing helps in detecting errors at an early stage and reduces overall development time and cost. The results confirm that all functions are operating as expected.

ValidationTesting

At the final stage of development, the system is tested as a complete unit to ensure that all modules are properly integrated. Validation testing ensures that the system meets user requirements and performs tasks correctly in real-world scenarios. It verifies whether the system fulfils its intended purpose and produces accurate outputs. This process helps in confirming that the developed system is reliable and meets the project objectives.

FunctionalTesting

Functional testing is conducted to ensure that each feature of the application works according to the defined requirements. In this project, features such as user login, bug creation, task management, comments, file uploads, and notifications are tested. The system is evaluated by providing sample inputs and comparing the actual results with expected outputs. This ensures that all features function correctly and meet user expectations.

Test Case Analysis of Bug Tracker System

Function Name	Test Results	Expected Output	Actual Output
User Login	Tested with valid and invalid credentials	User should login successfully or show error	System logs in user correctly or displays error
Create Bug	Tested with different inputs	Bug should be created and stored	Bug created and stored successfully
Update Bug	Tested by changing status	Bug status should update	Bug status updated correctly
Create Task	Tested with task details	Task should be created	Task created successfully
Assign Task	Tested by assigning to users	Task should be assigned	Task assigned correctly
Add Comment	Tested with comments	Comment should be displayed	Comment displayed successfully
Upload File	Tested with file upload	File should be uploaded	File uploaded successfully
Notification System	Tested on task assignment	Notification should be generated	Notification displayed correctly
Dashboard View	Tested for data display	Dashboard should show summary	Dashboard shows correct data

Table 6.2: User Registration Test

Test Case	1
Name of Test	User Registration Test
Input	Username: rani123 Email: rani@gmail.com Password: Rani@123
Expected Output	Registration Successful
Actual Output	Registration Successful
Result	Successful

Table 6.3: User Login Test

Test Case	2
Name of Test	User Login Test
Input	Username: rani123 Password: Rani@123 Password: Rani@123
Expected Output	Login Successful
Actual Output	Login Successful
Result	Successful

Table 6.4: Bug Creation Test

Test Case	3
Name of Test	Create Bug Test
Input	Title: Login Error Description: Unable to login Priority: High
Expected Output	Bug Created Successfully
Actual Output	Bug Created Successfully
Result	Successful

Table 6.7: File Upload Test

Test Case	6
Name of Test	File Upload Test
Input	File: screenshot.png
Expected Output	File uploaded successfully
Actual Output	File uploaded successfully
Result	Successful

Table 6.8: Comment Feature Test

Test Case	7
Name of Test	Add Comment Test
Input	Input: “Fixed the bug, great work!”
Expected Output	Comment displayed successfully
Actual Output	Comment displayed successfully
Result	Successful

8 Conclusion and Future Scope

Conclusion

The Bug Tracker and Task Management System has been successfully developed to deliver an effective solution for managing bugs and tasks in a well-structured manner. The system allows users to report bugs, assign tasks, and track their progress efficiently. It simplifies the handling of software issues and enhances coordination among team members.

The application ensures secure user authentication and supports various features such as bug creation, task management, comments, file attachments, and notifications. These functionalities improve collaboration and ensure that all activities are properly monitored. The dashboard provides a clear overview of system performance, enabling users to analyze and manage their work effectively.

The results obtained from testing indicate that the system performs reliably and meets the intended objectives. It reduces manual effort, increases productivity, and ensures better organization of data. Overall, the system is user-friendly, efficient, and suitable for real-world applications.

Future Scope

The system can be further enhanced by incorporating advanced features and improvements. Real-time communication capabilities such as chat functionality can be added to strengthen collaboration among users. Integration with external tools like GitHub can also be implemented to link bugs directly with code repositories.

The system can be expanded by developing mobile applications to provide easier accessibility. Advanced analytics and graphical reports can be introduced to offer deeper insights into bug trends and task performance. Role-based access control can be improved by adding more user roles and permissions.

Furthermore, features such as email notifications, automated reminders, and AI-based recommendations can be included to make the system more intelligent and efficient. These enhancements will improve the overall functionality and usability of the system in future developments.

9 Bibliography

- [1] Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2015.
- [2] Roger S. Pressman, *Software Engineering: A Practitioner's Approach*, 7th Edition, McGraw Hill, 2014.
- [3] Django Documentation [online]. Available: <https://docs.djangoproject.com/>
- [4] Python Documentation [online]. Available: <https://www.python.org/doc/>
- [5] Bootstrap Documentation [online]. Available: <https://getbootstrap.com/>
- [6] W3Schools Web Tutorials [online]. Available: <https://www.w3schools.com/>
- [7] GitHub Platform [online]. Available: <https://github.com/>
- [8] Stack Overflow Developer Community [online]. Available: <https://stackoverflow.com/>
- [9] Tutorialspoint Programming Tutorials [online]. Available: <https://www.tutorialspoint.com/>
- [10] GeeksforGeeks Programming Articles [online]. Available: <https://www.geeksforgeeks.org/>
- [11] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, Addison-Wesley, 2004.
- [12] E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- [13] Oracle Corporation, "JavaScript Documentation" [online]. Available: <https://developer.mozilla.org/>
- [14] SQLite Documentation [online]. Available: <https://www.sqlite.org/docs.html>
- [15] Mozilla Developer Network (MDN), "HTML and CSS Documentation" [online]. Available: <https://developer.mozilla.org/>

